



Atenção

- Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.
- O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.
- Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

O enunciado pede substituir a espera ocupada de `timer_sleep` por bloqueio de threads no `Pintos`. A entrega inclui o Alarm Clock, testes de prazo mínimo, esperas simultâneas, zero e valores negativos. A validação em base limpa aprovou sete testes. Foram exportados somente os 14 arquivos novos ou alterados, com `Makefile` e evidências separados.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Suspender a thread sem espera ocupada	OK	alarm-blocked: <code>THREAD_BLOCKED</code> .
Esperar pelo menos o prazo solicitado	OK	alarm-minimum: 100 ticks.
Tratar uma espera por thread	OK	alarm-single: PASS.
Tratar várias esperas por thread	OK	alarm-multiple: PASS.
Liberar todas as esperas vencidas no mesmo tick	OK	alarm-simultaneous: PASS.
Retornar imediatamente para tempo zero	OK	alarm-zero: 100 chamadas.
Aceitar durações negativas, incluindo <code>INT64_MIN</code>	OK	alarm-negative: PASS.
Compilar do zero e conferir os resultados	OK	make validar: sete PASS.

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
No Ubuntu/WSL, na raiz do repositório pint-0s:  
make clean  
make
```

Executar (exemplo):

```
make validar
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

- `src/devices/timer.c` - mantém a fila, bloqueia e desperta as threads.
- `src/tests/threads/` - testes de prazo, bloqueio, limites e concorrência.
- `Makefile`, `Make.config` e `Make.vars` - compilação, QEMU e validação.
- `evidencias.log` - resultados reais; logs brutos também preservados.

Decisões técnicas:

- Fila ordenada por prazo absoluto e semáforo por espera; a thread libera a CPU.
- Interrupções desativadas durante a inserção e o bloqueio protegem a fila. O nível anterior é restaurado no retorno.
- O timer libera todas as esperas vencidas. Zero e negativos retornam antes da fila; `TIMER_FREQ` e funções auxiliares foram preservados.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Espera bloqueante	Fila ordenada e semáforo por espera.	Eliminar o laço com <code>thread_yield</code> .
S2	Prazos e limites	Testes de duração, concorrência e zero/negativos.	Detectar retornos antecipados.
S3	Estado e base limpa	Verificar <code>THREAD_BLOCKED</code> e compilar só as alterações sobre a base original.	Testes de prazo não comprovavam o bloqueio real.

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do `evidencias.log`, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Adicionar fila e teste de prazo.	Falha	Demonstração: <code>duration/2</code> .	17/09	6f19048; log: 100 -> 50 ticks.
2	S2	Corrigir o prazo completo.	OK	-	19/09	0a0afd5; alarm-minimum.
3	S2	Liberar só uma espera por tick.	Falha	Demonstração: break indevido.	20/09	Log concorrência-falha.
4	S2	Restaurar todas as liberações.	OK	-	20/09	0097ae7; simultaneous PASS.
5	S2	Rejeitar duração zero.	Falha	Demonstração: <code>ASSERT > 0</code> .	20/09	Log limites-falha; PANIC.
6	S2	Tratar zero e negativos; testar <code>INT64_MIN</code> .	OK	-	20/09	b7fdf21; logs limites-corretos.
7	S3	Detectar <code>thread_yield</code> e restaurar semáforo.	Falha / OK	Demonstração de espera ocupada detectada.	20/09	2154e72; Figura 1; alarm-blocked.
8	S3	Compilar as alterações sobre base limpa.	OK	-	20/09	fa558f1; Figura 2; sete PASS.

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do `evidencias.log` com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com `date`, `whoami` e `pwd`, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

O make validar recompilou as alterações sobre uma base limpa: sete testes PASS. Os logs vieram dos arquivos `.output/result` e de redirecionamento, não de uma sessão completa com `script`. As falhas demonstrativas foram restauradas antes da validação final. Repositório: github.com/danduleba/pintos-0s (login da entrega: dad). Commits publicados (links):

[d33650f](#)

[6f19048](#)

[0a0afd5](#)

[0097ae7](#)

[b7fdf21](#)

[2154e72](#)

[fa558f1](#)

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.

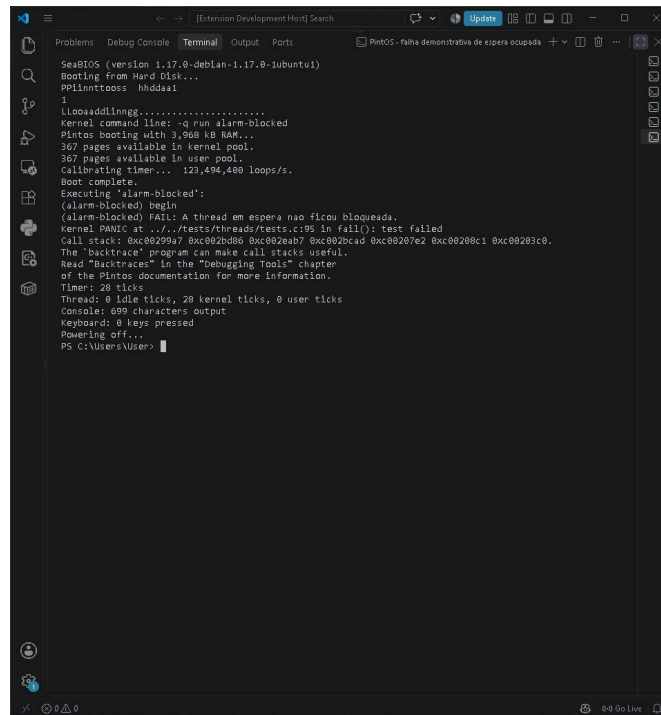


Figura 1 - Falha demonstrativa: alarm-blocked detecta espera ocupada.

- Validação final: teste passando / saída correta.

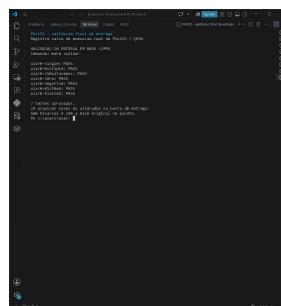


Figura 2 - Validação em base limpa: sete testes PASS; 14 arquivos alterados.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Usei IA para ajudar na instalação do Pintos e para saber os códigos (hashes) dos commits no GitHub, e no relatório para inserção da série de testes do Pintos.

Série de testes do Pintos - 20/09/2026 às 14:36 (Brasília)

35/35 execuções aprovadas: sete testes em cinco rodadas. Primeira rodada: make validar (compilação limpa). Demais: make testar-alarmes, removendo .output, .result e .errors antes de cada rodada. Conferidos os arquivos .result e os códigos de saída.

alarm-single	5/5 PASS	Uma espera por thread.
alarm-multiple	5/5 PASS	Esperas repetidas por thread.
alarm-simultaneous	5/5 PASS	Threads com o mesmo prazo.
alarm-zero	5/5 PASS	100 chamadas com zero por execução.
alarm-negative	5/5 PASS	-1, -100 e INT64_MIN; interrupções preservadas.
alarm-minimum	5/5 PASS	Espera de pelo menos 100 ticks.
alarm-blocked	5/5 PASS	Estado THREAD_BLOCKED durante a espera.

Código testado: fa558f1. Ubuntu/WSL e QEMU. Logs: relatorio/logs/serie-20260920T173421Z/, com saídas por rodada e resultados.json. São sete casos repetidos, não 35 casos diferentes. As figuras da página anterior registram a validação anterior. Prioridades e MLFQS não fazem parte desta bateria.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

A fila ordenada simplificou o despertar das threads, e o semáforo garantiu bloqueio real. Os testes mostraram que cumprir o prazo não basta: uma espera ocupada também pode passar nos testes de tempo. O teste alarm-blocked detectou essa diferença. Zero e negativos precisam retornar antes da fila. A validação em base limpa conferiu os arquivos exportados.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	sim
Rodei os testes e evidencias.log foi gerado	sim
Tenho prints obrigatórios	sim
Testei pelo menos um caso limite e um caso inválido	Limites: sim; negativos são válidos.
Preenchi a seção de uso de IA	sim
Revisei o relatório e removi frases genéricas / vazias	sim

10. Se eu tivesse mais 2 horas

Eu ampliaria a quantidade de threads e repetiria os testes simultâneos sob cargas diferentes. Também verificaria durações positivas próximas do limite de int64_t e o possível overflow da soma com os ticks atuais. Conferiria a compilação em outra instalação Linux. Esses testes adicionais ainda não foram executados.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.